

## Lecture 6: Java RMI (Remote Method Invocation)

### 1. RMI Overview

#### ✓ Ce este RMI (Remote Method Invocation)?

RMI este o tehnologie Java care permite unui program care rulează într-o JVM (client) să **apeleze metode ale unui obiect aflat într-o altă JVM (server)** — ca și cum ar fi un obiect local.

#### 🎯 Scop:

- Permite **apeluri de metode de la distanță** între aplicații Java.
- Facilitează dezvoltarea de **aplicații distribuite** în mod natural și OOP-style.

#### ☀️ Beneficiu principal:

- **Ascunde complexitatea rețelei**, cum ar fi sockets, serializare sau protocoale, în spatele unor **apeluri de metode obișnuite**.

---

### 2. RMI Architecture – Componente

#### ♦ 1. RMI Registry

- Este un **serviciu de denumire** care rulează de obicei pe portul **1099**.
- Serverul **înregistrează (bind)** obiectele remote cu un **nume logic**.
- Clientul **face lookup** folosind acel nume pentru a obține un **stub (proxy)** către obiectul serverului.

---

#### ♦ 2. Stub și Skeleton

| Componentă                                       | Rol principal                    | Detalii                                                                                                    |
|--------------------------------------------------|----------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Stub</b> (pe client)                          | Proxy local al obiectului remote | - Implementează interfața remote.<br>- <b>Serializează (marshals)</b> parametrii și îi trimite prin rețea. |
| <b>Skeleton</b> (pe server – doar în Java vechi) | Dispatcher care primește apelul  | - <b>Deserializează (unmarshals)</b> datele primite.<br>- Apelează metoda reală și returnează rezultatul.  |

📌 **Notă:** Din Java 5, skeleton-ul **nu mai este generat**, iar apelurile sunt rutate direct prin **dynamic proxies**.

---

### ♦ 3. JRMP Protocol

- Protocolul standard folosit de RMI: **Java Remote Method Protocol**.
  - Funcționează **doar între aplicații Java** (Java-to-Java).
- 

### ♦ 4. Security Manager și Policy

- Se setează un **SecurityManager**, de obicei un **RMISecurityManager**, care:
    - **Aplică reguli de securitate** definite într-un fișier **java.policy**.
    - Poate permite sau restricționa **descărcarea de clase remote**.
- 

## 3. Pașii dezvoltării RMI

### Server-side

#### 1. Definește Interfața Remote:

```
public interface SampleServerInterface extends Remote {  
    int sum(int a, int b) throws RemoteException;  
}
```

#### 2. Implementează Clasa Remote:

```
public class SampleServerImpl extends UnicastRemoteObject implements  
SampleServerInterface {  
    public SampleServerImpl() throws RemoteException { super(); }  
    public int sum(int a, int b) throws RemoteException {  
        return a + b; }  
}
```

#### 3. Scrie Main-ul Serverului:

```
System.setSecurityManager(new RMISecurityManager());  
SampleServerImpl server = new SampleServerImpl();  
Naming.rebind("rmi://localhost:1099/SAMPLE-SERVER", server);
```

#### 4. Compilează și Generează Stubs:

```
javac *.java
rmic SampleServerImpl # (necesar doar în versiunile mai vechi)
```

## 5. Rulează Serviciile:

```
start rmiregistry # lansează registry-ul
java ServerMain  # pornește serverul
```

---

## Client-side

### 1. Setează SecurityManager:

```
System.setSecurityManager(new RMISecurityManager());
```

### 2. Caută Obiectul Remote:

```
SampleServerInterface stub = (SampleServerInterface)
Naming.lookup("rmi://localhost:1099/SAMPLE-SERVER");
```

### 3. Apelează Metoda Remote:

```
int result = stub.sum(1, 2);
System.out.println("1 + 2 = " + result);
```

---

## 4. Detalii importante (Gotchas)

- Toate metodele remote trebuie să declare `throws RemoteException`.
- Parametrii și valorile returnate trebuie să fie **Serializable**.
- Portul default este **1099**, dar poate fi schimbat (clientul trebuie să știe portul corect).
- **Stubs dinamice**: JVM-urile moderne generează automat stubs — `rmic` nu mai e necesar, dar este bine să știi despre el.
- Probleme de **rețea/firewall** pot bloca apelurile — trebuie deschise porturile pentru registry și pentru obiectele remote.
- **Class loading**: Clientul trebuie să aibă acces la interfața și clasa stub. Pentru a permite descărcarea dinamică, se setează `codebase` în policy.

---

## 5. Puncte importante pentru examen/interviu

### Termeni esențiali:

- RMI
- Stub & Skeleton
- Marshalling/Unmarshalling
- JRMP
- RemoteException

### Use-Cases:

- Aplicații distribuite între JVM-uri (e.g. module EJB, aplicații JEE).

### Analiză logică:

Ce se întâmplă când clientul apelează `stub.method()`?

1. Se face **lookup** în registry.
2. Stub-ul **marshalează** parametrii.
3. Serverul primește apelul, **unmarshalează** și apelează metoda reală.
4. Rezultatul este **marshale-at** înapoi și returnat clientului.

### Detalii de configurare:

- `SecurityManager`, `java.policy`, `rmiregistry`, `Naming.rebind/lookup`
- Porturi, excepții, acces la clase.

## Lecture 8: SOA vs. REST s HTTP 1 vs. 2; Open MPI vs. OpenMP; IaaS Cloud s OpenMPI on AWS EC2

### 1. SOA vs. REST

#### SOA (Service-Oriented Architecture)

- **Definiție:** Stil arhitectural unde serviciile software se comunică prin contracte standardizate, de obicei cu **SOAP** peste HTTP și descrierea interfețelor prin **WSDL**.
- **Caracteristici:**
  - Contract rigid: fiecare metodă, tip de mesaj și structură de date este **explicit definită**.
  - Folosește **XML**, cu multe extensii (WS-Security, WS-Reliability etc).
  - Potrivit pentru **sisteme mari, enterprise**, unde ai nevoie de tranzacții sigure, audit, securitate.

## REST (Representational State Transfer)

- **Definiție:** Stil arhitectural care folosește direct conceptele HTTP – fiecare **resursă** este identificată printr-un **URI**, iar acțiunile se fac prin **verbe HTTP** (GET, POST, PUT, DELETE).
- **Caracteristici:**
  - **Stateless** – fiecare cerere conține toate informațiile.
  - Datele sunt în **JSON** (preferat), dar pot fi și XML sau text.
  - REST adevărat folosește coduri de stare HTTP (ex: 404, 201).
  - Urmează principiul HATEOAS: răspunsul poate conține linkuri către alte acțiuni.

### Diferență esențială:

- **SOA** = orientat pe **operații** (RPC-like)
  - **REST** = orientat pe **resurse** (nume: utilizator, produs, etc.)
- 

## 2. HTTP/1.1 vs. HTTP/2

| Caracteristică             | HTTP 1.1                           | HTTP 2.0                                          |
|----------------------------|------------------------------------|---------------------------------------------------|
| <b>Framing</b>             | Textual                            | Binare (HEADERS, DATA, etc.)                      |
| <b>Multiplexing</b>        | O singură cerere per conexiune TCP | Mai multe cereri pe o conexiune, fără blocare     |
| <b>Header Compression</b>  | Nu                                 | Da, cu HPACK                                      |
| <b>Server Push</b>         | Nu                                 | Serverul poate trimite proactiv resurse           |
| <b>TLS handshake</b>       | Se face separat                    | Se face cu <b>ALPN</b> în timpul TLS              |
| <b>Performanță</b>         | Sharding, concatenare              | Nu mai sunt necesare; conexiune unică performantă |
| <b>Cerere de conexiune</b> | HTTP sau HTTPS                     | Doar HTTPS (TLS obligatoriu)                      |

🔧 **Gotcha:** Server Push trebuie folosit cu grijă – dacă trimiți ceva ce clientul are deja în cache, pierzi performanță.

---

### 3. OpenMPI vs. OpenMP

| Aspect               | Open MPI (distribuit)                                   | OpenMP (shared memory)                                  |
|----------------------|---------------------------------------------------------|---------------------------------------------------------|
| <b>Model</b>         | Proces per nod, comunicație prin mesaje (MPI_Send/Recv) | Thread-uri într-un singur proces (shared memory)        |
| <b>Scalabilitate</b> | Scalabil pe mii de noduri, rețea                        | Limitat la un singur nod cu mai multe core-uri          |
| <b>Cod simplu</b>    | Mai greu: ai de gestionat rank-uri, mesaje              | Mai ușor: adaugi <code>#pragma omp</code> la for-uri    |
| <b>Paralelizare</b>  | Manuală, la nivel de procese                            | Automată, la nivel de loop-uri                          |
| <b>Sincronizare</b>  | Colective (ex: MPI_Bcast), blocking                     | Implicită (barieră la sfârșit de <code>omp for</code> ) |
| <b>Overhead</b>      | Costuri de rețea, latență                               | Probleme de cache coherence, overhead thread-uri        |

#### 🧠 OpenMP Gotchas:

- Variabilele sunt **shared** implicit – folosește `private`, `reduction`.
- Planificarea loop-urilor: `schedule(static, dynamic, guided)`.
- I/O: Trebuie sincronizat manual (nu e automat safe între thread-uri).

#### 📦 MPI Gotchas:

- Funcțiile sunt **blocking** (ex: `MPI_Recv` așteaptă până primește mesaj).
- `MPI_Send/Recv` trebuie să se potrivească: **tip, tag, dimensiune, communicator**.
- Funcțiile colective trebuie apelate de toți rank-urile (altfel = deadlock).

---

### 4. IaaS Cloud & Open MPI pe AWS EC2

#### ☁️ IaaS (Infrastructure as a Service):

- Oferă acces la resurse virtuale (mașini, rețele, stocare).
- Exemple: **AWS EC2**, Azure VMs, Google Compute Engine.

#### ⚙️ Open MPI pe AWS EC2:

1. **Instanțe EC2:** Lansate cu aceeași imagine (Ubuntu, Amazon Linux etc.), preferabil din seria **C5, HPC-optimized**.
2. **Networking:** Folosește **placement group (cluster mode)** pentru latență mică.
3. **Configurare:**
  - Instalezi **OpenMPI** pe toate instanțele.
  - Configurezi SSH fără parolă (folosind chei).

#### Lansare Job MPI:

```
mpirun -np 4 --hostfile myhosts.txt ./program
```

4. **Optimizare costuri:**
  - Poți folosi **spot instances** (mai ieftine, dar pot fi întrerupte).

 Resurse utile:

- Ghid setup rapid MPI: Glenn Klockwood

---

## Exam Focus Summary

| Concept                   | Ce să reții                                                                             |
|---------------------------|-----------------------------------------------------------------------------------------|
| <b>SOA vs. REST</b>       | SOAP = operații, XML; REST = resurse, JSON, HTTP verbs                                  |
| <b>HTTP 1.1 vs 2</b>      | HTTP/2 = binar, multiplexat, rapid, TLS obligatoriu                                     |
| <b>OpenMPI vs. OpenMP</b> | MPI = multi-node, mesaj; OpenMP = shared memory, pragmas                                |
| <b>Cod analizat</b>       | <code>#pragma omp for schedule(...)</code> , <code>MPI_Send/Recv</code> , HTTP/2 frames |
| <b>AWS Config</b>         | EC2 tipuri, SSH, hostfile, <code>mpirun</code> , securitate porturi                     |

## Lecture 10: JMS (Java Message Service) s Apache Kafka

### 1. JMS (Java Message Service) – Overview

JMS este o **API standard Java** pentru **Message-Oriented Middleware (MOM)**, adică un sistem care permite aplicațiilor să comunice prin mesaje, **fără să fie conectate direct între ele**.

#### Modele de mesagerie:

- **Point-to-Point (Queue):**
  - 1 producător → 1 consumator.
  - Mesajele merg într-o coadă (Queue), și **fiecare mesaj e consumat o singură dată**.

- **Publish-Subscribe (Topic):**
  - 1 producător → mai mulți abonați.
  - Toți abonații activi primesc **câte o copie** a fiecărui mesaj.



### Moduri de consumare:

- **Synchronous:** `receive()` → aștepti blocant până vine mesajul.
- **Asynchronous:** `MessageListener.onMessage()` → sistemul te notifică automat când sosește mesajul.



### Durabilitate:

- **Non-Durable:** primești mesaje doar cât ești conectat.
- **Durable:** brokerul reține mesajele și dacă ești offline; le primești când revii.



---

## 2. JMS Architecture & Administered Objects



### Obiecte administrate (administered objects):

- **ConnectionFactory** – creat prin JNDI, stabilește conexiunea cu brokerul JMS.
- **Destination (Queue/Topic)** – obiecte logice tot prin JNDI, care ascund detaliile fizice (ex: nume, locație).



### JMSContext (JMS 2.0):

- API mai simplu, cu `try-with-resources`, nu mai ai nevoie de `Connection`, `Session`, `Producer` separat.



### Tipuri de mesaje:

- `TextMessage`, `ObjectMessage`, `BytesMessage`, etc.



### Request-Reply Pattern:

- **JMS nu are request-reply nativ**, dar îl poți simula astfel:
  1. Creezi o coadă temporară (`createTemporaryQueue()`).
  2. Setezi `setJMSReplyTo(tempDest)`.
  3. Consumerul răspunde în acea coadă.



---

## 3. JMS Basic Send/Receive – Cod exemplu



### Trimitere mesaj:



```
Context ctx = new InitialContext();
ConnectionFactory cf = (ConnectionFactory) ctx.lookup("jms/CF");
Queue queue = (Queue) ctx.lookup("jms/OrderQueue");

try (JMSContext jctx = cf.createContext();
     JMSProducer producer = jctx.createProducer()) {
    TextMessage msg = jctx.createTextMessage("New Order: 12345");
    producer.send(queue, msg);
}
```

### Consum asincron:

```
try (JMSContext jctx = cf.createContext()) {
    JMSConsumer consumer = jctx.createConsumer(queue);
    consumer.setMessageListener(m -> {
        try {
            System.out.println("Received: " + ((TextMessage)m).getText());
        } catch (JMSException e) { /* handle */ }
    });
}
```

**!! Notă:** întotdeauna tratează `JMSException` și închide conexiunile corespunzător.

---

## 4. Tranzacții & Fiabilitate

### Tranzacții:

- **CMT (Container-Managed):** EJB/JTA gestionează tranzacțiile.
- **BMT (Bean-Managed):** manual cu `UserTransaction`.

### Acknowledgement Modes:

- `AUTO_ACKNOWLEDGE`: automat.
- `CLIENT_ACKNOWLEDGE`: tu confirmi când ai procesat.
- `DUPS_OK_ACKNOWLEDGE`: permite duplicări.
- `SESSION_TRANSACTED`: integrat în tranzacție.

### QoS (Quality of Service):

- **Persistent vs. Non-Persistent:** mesaje salvate sau volatile.
  - **Message Selectors:** filtrezi după proprietăți (ex: prioritate = 'urgent').
-

## 5. Apache Kafka Overview

Kafka este o **platformă distribuită de mesagerie, scalabilă și foarte rapidă**, pentru evenimente/loguri în timp real.

### Concepte de bază:

- **Broker**: server Kafka.
- **Topic**: flux de mesaje.
- **Partition**: fiecare topic e împărțit în mai multe partitii (scalabilitate + paralelism).
- **Offset**: indexul fiecărui mesaj în partitie.

### API:

- **Producători**: trimit mesaje (poți controla partajarea).
- **Consumatori**: citesc mesaje, gestionează offset-ul, pot face parte dintr-un **consumer group** pentru load balancing.

### Durabilitate & Scalabilitate:

- Mesajele sunt salvate pe disc și replicate între brokeri.
- **Consumatorii lenți nu afectează producătorii** (pull-based model).

### Ecosistem:

- **Kafka Connect**: conectează sisteme externe.
- **Kafka Streams**: procesare de date în flux, nativ în Kafka.

---

## 6. JMS vs Kafka – Diferențe cheie

| Aspect        | JMS            | Kafka                  |
|---------------|----------------|------------------------|
| Model         | Push-based     | Pull-based             |
| Scalabilitate | Limitată       | Foarte bună (partiții) |
| Durabilitate  | Opțională      | Implicită              |
| Ordine        | Per-destinație | Per-partiție           |

|                         |                                         |                                           |
|-------------------------|-----------------------------------------|-------------------------------------------|
| <b>Garantie livrare</b> | At-most-once, transacted = exactly-once | Exactly-once (cu idempotent + transacții) |
| <b>Complexitate API</b> | Configurație mare                       | API simplu, ecosistem bogat               |

**Atenție!** Kafka implică mai multă muncă pe partea de **evoluție a schemelor (Schema Registry)** și **rebalansare de consumatori**.

---

## 7. Ce trebuie să știi pentru examen:

### DEFINIȚII:

- JMS, MOM, Queue vs. Topic, sync vs. async, JMSReplyTo, broker, partition, offset.

### ARHITECTURĂ:

- JMS: ConnectionFactory, JMSContext, Destination.
- Kafka: Broker, Consumer Group.

### ANALIZĂ COD:

- JMS: de la JNDI → JMSContext → producer/consumer → listener.
- Kafka: `consumer.poll()` → fetch → procesare → commit offset.

### USE CASES:

- **JMS**: când vrei mesagerie tranzacțională în JEE (ex: comenzi).
- **Kafka**: când ai nevoie de loguri, streamuri, volum mare (ex: analytics în timp real).

### DETALII IMPORTANTE:

- JMS: `message selectors`, `acknowledgement modes`, `durable subscriptions`.
- Kafka: `retention policy`, `log compaction`, `exactly-once delivery`.

## Lecture 11: EJB 3.x s Akka Actors

### ◆ Part I – EJB 3.x (Enterprise Java Beans)

## 1. Ce este EJB și ce oferă containerul

- **EJB = componente Java enterprise** gestionate de un container (ex. WildFly, GlassFish).
  - Containerul oferă:
    - **Transacții** (fără să le gestionezi manual)
    - **Securitate** (control acces)
    - **Pooling** (reutilizare obiecte pentru performanță)
    - **Acces Remote** (poți apela bean-ul de pe alt server)
  - Bean-urile comunică cu containerul prin:
    - **JNDI/ENC** pentru a obține resurse (JDBC, alte bean-uri).
    - **Callback-uri**: metode speciale ca `@PostConstruct`, `@PreDestroy` sunt apelate la evenimente din ciclul de viață.
    - **EJBContext** (ex: `SessionContext`) – pentru info despre runtime, utilizator, tranzacție curentă.
- 

## 2. Tipuri de Beans în EJB 3.x

### ✓ Session Beans

- **Stateless**
  - Nu reține stare între apeluri.
  - Bun pentru servicii simple, reutilizabile.
  - Se pot reutiliza prin pool; metodele trebuie să fie *reentrant* (să nu depindă de stare internă).
- **Stateful**
  - Reține stare pentru un client specific (ex: coș de cumpărături).
  - Necesită gestiune pentru **passivation/activation** (scriere pe disc când e inactiv, apoi încărcare).

### ✓ Message-Driven Beans (MDBs)

- Răspund la mesaje JMS (asincron).
- Ca și stateless, dar *atenție*: pot primi mesaje în altă ordine decât trimise → trebuie să fie **idempotente**.

### ✓ Entity Beans (acum înlocuite de JPA)

- Reprezintă date persistente.
  - Două tipuri:
    - **CMP**: containerul gestionează maparea către DB (ex: JPA cu `@Entity`, `@Id` etc.).
    - **BMP**: tu scrii codul pentru persistare (ex: JDBC).
-

### 3. Diferențe EJB 3.x vs. 2.x

- EJB 3.x e mai ușor:
    - POJO + **anotări** (`@Stateless`, `@EJB`, `@PersistenceContext`) în loc de XML.
    - **Injection** de dependențe în loc de JNDI.
    - Interfețe remote doar dacă e nevoie de apeluri la distanță.
    - Simplificare a fișierelor de configurare (`ejb-jar.xml` minim).
- 

### 4. Exemplu de Stateless Bean

```
@Stateless
public class HotelClerkBean implements HotelClerk {
    @PersistenceContext
    private EntityManager em;

    public Reservation book(Guest g, Room r) {
        Reservation res = new Reservation(g, r);
        em.persist(res);
        return res;
    }
}
```

- EJB container se ocupă de:
    - Crearea bean-ului
    - Injectarea `EntityManager`
    - Gestiunea tranzacției (implicit `REQUIRED`)
- 

### 5. Ciclul de viață și capcane

- **Stateless:** `@PostConstruct` → metode → `@PreDestroy`
  - **Stateful:**
    - Inițializare: `@PostConstruct`
    - Când devine inactiv: `@PrePassivate`
    - Când e reactivat: `@PostActivate`
    - Finalizare: `@PreDestroy`
  - **MDBs:**
    - `@PostConstruct` → `onMessage()` → `@PreDestroy`
    - Concurență ridicată – **idempotent** înseamnă același rezultat indiferent de câte ori procesezi mesajul.
-

## Remote Access & Transactions

- **Tranzacții:** recomandat container-managed (automat).
  - **Remote Beans:** folosesc RMI/IIOP sub capotă, cu proxy-uri.
- 

## ◆ Part II – Akka Actors

### 1. Modelul Actor

- Actor = unitate de calcul de bază:
    - Are **stare proprie** (nu se partajează).
    - Primește **mesaje asincron** printr-o coadă (mailbox).
    - Poate:
      - Crea alți actori
      - Trimite mesaje
      - Să își **schimbe comportamentul** în runtime
- 

### 2. Actor System și Adresare

- **ActorSystem** = container care gestionează actorii și firele de execuție.
  - **Adrese:**
    - Local: `system.actorOf(Props.create(MyActor.class), "myActor")`
    - Remote: prin config (`akka.remote.netty.tcp`) →  
`akka://sys@host:port/user/myActor`
- 

### 3. Remote Deployment & Clustering

- **Remote deployment:** poți specifica că un actor rulează pe un alt nod.
  - **Clustering:**
    - Distribuie munca între mai multe noduri.
    - Reacționează dinamic la noduri adăugate/scoase.
    - Crește disponibilitatea aplicației (fault-tolerance).
- 

### 4. Reactive & Event-Driven

- Model **non-blocking** și **orientat pe evenimente**.
- Bun pentru aplicații:
  - Concurente
  - Microservicii
  - Streaming

- IoT / real-time
- **Supraveghere:** actorul părinte decide ce face când copilul eșuează (restart, stop, etc.).

## 5. Comparativ: EJB vs Akka

| Aspect                 | EJB Session Beans            | Akka Actors                             |
|------------------------|------------------------------|-----------------------------------------|
| <b>State</b>           | Gestiune de container        | Stare în actor (memorie locală)         |
| <b>Concurență</b>      | Apeluri sincrone             | Mesaje asincrone                        |
| <b>Scalare</b>         | Verticală (clustering app)   | Orizontală (distribuire actori)         |
| <b>Fault-tolerance</b> | Tranzacții container         | Supervizare ierarhică                   |
| <b>Use Cases</b>       | Business logic cu tranzacții | Streaming, IoT, realtime, microservicii |

## ✓ Puncte esențiale pentru examen

- Diferențe: **stateless vs. stateful**, **MDB**, **CMP vs BMP**, **actor**, **mailbox**, **supervision**.
- **Callback-uri** importante:
  - `@PostConstruct`, `@PreDestroy`, `@PrePassivate`, `@PostActivate`, `onMessage()`
- **Analiză de cod:**
  - Înțelege fluxul: injecție, tranzacții, tratarea mesajelor.
- **Capcane:**
  - MDB: mesaje în afara ordinii → idempotent!
  - Stateful: atenție la starea prea mare (nu poate fi serializată ușor)
  - Akka: actorii **nu partajează stare**; comunică doar prin mesaje **imutabile**.

## Lecture 12: Distributed Architectures s Protocols Overview

### 1. Architectural Styles (Stiluri Arhitecturale)

#### a) Client–Server

- Model tradițional cu două niveluri: clientul trimite cereri, serverul răspunde.
- Serverul gestionează atât logica de prezentare, cât și cea de date.
- **!** *Problemă:* legătură strânsă client-server → scalabilitate redusă.

#### b) n-Tier (ex. 3-Tier Architecture)

- Trei niveluri separate:
  1. **UI** (interfață grafică)
  2. **Business logic**
  3. **Date** (bază de date)

- Avantaj: separare clară a responsabilităților.
- **MVC**: Modelul influențează View direct (ex: Spring MVC).
- **MVP**: decuplează UI de logică printr-un „Presenter”.

#### c) ESB (Enterprise Service Bus)

- Bus de comunicație între servicii diferite (middleware).
- Oferă: rutare, transformare, orchestrare.
- **!** *Problemă*: devine Single Point of Failure dacă nu e clusterizat.

#### d) SOA & Microservices

- Aplicația este împărțită în servicii mici, independente.
  - Comunică prin HTTP/REST sau SOAP.
  - **!** *Probleme*: latență, consistență eventuală, complexitate operațională.
- 

## 2. Peer-to-Peer (P2P)

#### a) Pure P2P

- Toți nodurile sunt egale, comunică direct (ex: BitTorrent).
- **!** *Dificultăți*: NAT traversal, descoperire noduri, securitate.

#### b) Hybrid P2P

- Folosește un nod central (tracker) sau super-peers.
- Mai eficient, dar nu complet descentralizat.

#### c) Use Cases

- Sharing fișiere, rețele blockchain, aplicații descentralizate.
- 

## 3. Grid & HPC (High Performance Computing)

#### a) Grid Middleware

- Exemple: Globus, Condor, gLite, BOINC.
- Folosit pentru partajarea resurselor între organizații.

#### b) Caracteristici

- Rețele mari, resurse eterogene, securitate cu GSI.
- **!** *Probleme*: fairness în job scheduling, managementul replicilor.

#### c) MapReduce

- Model master-worker (ex: Hadoop).



- Toleranță la erori, localitate a datelor.
- Include: combiner, speculative execution.

---

## 4. Cloud Service Models

| Model | Ce oferă                                 | Exemplu           |
|-------|------------------------------------------|-------------------|
| IaaS  | Mașini virtuale, rețele, stocare         | AWS EC2           |
| CaaS  | Management de containere                 | Kubernetes        |
| PaaS  | Runtime complet cu tooluri de dezvoltare | Google App Engine |
| SaaS  | Aplicație gata de folosit                | Salesforce, Gmail |

! *Probleme:* vendor lock-in, costuri, securitate multi-tenant.

---

## 5. Protocols & Communication Platforms

### a) Transport + Socket APIs

- **TCP:** fiabil, lent (cu confirmări).
- **UDP:** rapid, fără garanție de livrare.

### b) RPC (Remote Procedure Call)

- **gRPC:** peste HTTP/2, rapid și eficient.
- **CORBA, DCOM:** limbaj-agnostic, folosește IDL-uri.
- **Java RMI:** RPC specific Java (acoperit în Lecture 6).

### c) Message-Oriented Middleware (MOM)

- Exemple: JMS, RabbitMQ, Kafka.
- **Pattern-uri:**
  - Queue (1 consumer), Topic (broadcast).
  - Push vs. Pull.
  - Idempotent messages, *exactly-once delivery*.

### d) Web Services

- **SOAP:** bogat în capabilități (QoS), dar foarte verbos.
- **REST:** simplu, eficient, dar necesită un design atent (hypermedia).

### e) P2P Protocols

- Ex: BitTorrent, JXTA, Ethereum.
- ⚠ Folosesc **DHT** (ex: Kademia) pentru descoperirea nodurilor.

## f) Agent & Actor Systems

- **JADE**: platformă pentru agenți (FIPA).
  - **Akka**: actori reactivi, model bazat pe mesaje (→ Lecture 11).
- 

## 6. Exam Focus – Ce să reții pentru examen

### ✓ Definiții importante:

- Client–Server vs. n-Tier vs. P2P vs. Grid vs. Cloud
- IaaS, PaaS, SaaS, CaaS
- SOAP vs. REST
- TCP vs. UDP

### ✓ Cazuri de utilizare:

- File sharing → P2P
- Streaming/event-driven → Microservices
- HPC → Grid
- Integrare servicii → ESB

### ✓ Trade-offs între protocoale:

- **TCP**: fiabil, dar mai lent.
- **UDP**: rapid, dar nesigur.
- **RPC**: tight coupling.
- **MOM**: loose coupling, mai flexibil.

### ✓ Gotchas:

- MVC ≠ 3-Tier (diferență de topologie).
- Grid: probleme cu securitatea și managementul datelor.
- DHT: sub/supra-partiționare duce la dezechilibru.
- Cloud: costuri și securitate în scenarii multi-tenant.